UCAM · INGENIERÍA INFORMÁTICA

Desarrollo de Aplicaciones Distribuidas I

Apuntes completos del primer parcial

Eclipse · Hilos · Sockets · IPC · HTTP · FTP · Plantillas de examen

Índice

1 Computación distribuida (Tema 1.1) 2

2 Comunicación entre procesos - IPC (Temas 1.2 y 2.1) 3

3 Eclipse: crear, ejecutar y depurar proyectos 5

4 Hilos de ejecución en Java 9

5 Sockets (Tema 2.2) 16

6 Protocolos de comunicación: HTTP y FTP (Tema 2.3) 20

7 El servidor multihilo del examen: receta, plantillas y ejemplos resueltos 25

8 Errores típicos que caen en el examen 35

9 Glosario rápido 36

10 Preguntas de teoría con respuesta modelo 38

Cómo usar estos apuntes

Están pensados para que **no se te escape nada**: cada tema empieza explicando la idea con palabras sencillas, después se ve la teoría tal y como la da el profesor, y al final tienes ejemplos resueltos y los errores típicos. Lo marcado como **«Extra»** no está en las diapositivas pero ayuda a entender lo que sí cae.

Parte del examen	Peso	Qué capítulos mirar
Preguntas cortas (teoría)	3 puntos	1, 2, 5 y 10 (definiciones, HTTP, ventajas, hilo, socket)
Preguntas de programación (qué falla en este código)	2 puntos	4 (hilos), 5 (sockets) y 8 (errores típicos)
Desarrollo: escribir el hilo de un servidor con sockets	5 puntos	3 (Eclipse), 6 (receta y plantillas) y 7 (ejemplos resueltos)

> **Orden recomendado**: 1 → 2 → 3 (Eclipse, para poder probar todo lo que sigue) → 4 → 5 → 6 → 7 → 8. Los capítulos 9 (glosario) y 10 (preguntas con respuesta modelo) son para repasar el día antes.

1 Computación distribuida (Tema 1.1)

La idea en una frase

Antes, un programa corría en **un solo ordenador** con **una sola CPU** (modelo *monoprocesador*). Hoy muchos programas se reparten entre **varios ordenadores conectados por red** que se ponen de acuerdo para hacer una tarea. Eso es la **computación distribuida**.

> **Sistema distribuido**: conjunto de computadores **independientes**, interconectados a través de una **red**, capaces de **colaborar** para realizar una tarea.

¿Qué significa «independientes»? Que **no comparten memoria ni espacio de ejecución**: cada uno tiene su RAM, su sistema operativo y sus procesos. Por eso, la única forma de que colaboren es **mandarse mensajes por la red** (eso es la comunicación entre procesos, capítulo 2).

> **Extra · ejemplos cotidianos**: cuando abres WhatsApp, tu móvil (cliente) habla con servidores de WhatsApp; al pagar online, tu navegador, el servidor de la tienda y el del banco colaboran. Ninguno ve la memoria de los otros: solo se mandan mensajes.

Vocabulario que debes dominar

Término	Significado
Monoprocesador	Un único procesador (el modelo anterior).
Sistema distribuido	Conjunto de ordenadores independientes conectados por red que colaboran para un fin.
Computación distribuida	La computación que se lleva a cabo en un sistema distribuido.
Servicio de red	Servicio que ofrece un programa especial (el **servidor**) en la red. Ej.: WWW, FTP, POP3 (correo).
Aplicación de red	Aplicación para usuarios finales que se ejecuta en ordenadores conectados por la red (el **cliente**, p. ej. un navegador).
RFC	*Request For Comments*: documentos donde se describen los protocolos de Internet (HTTP es el RFC 1945, FTP el RFC 959).

Un poco de historia

- **Años 60**: primera conexión para compartir ficheros.

- **1972**: primer correo electrónico, en ARPANET.
- **1985**: File Transfer Protocol (FTP).
- **Finales de los 80**: World Wide Web (WWW).

Ventajas e inconvenientes

Ventajas	Por qué	Inconvenientes
Económica	Los computadores y el acceso a red son baratos.	**Múltiples puntos de fallo**: si cae una máquina o la red, parte del sistema deja de funcionar.
Compartición de recursos	Varios usuarios usan los mismos datos, impresoras, servicios…	**Seguridad**: más equipos y más comunicaciones = más puntos por donde atacar.
Escalabilidad	Si crece la carga se añaden más máquinas.	
Tolerancia a fallos	Si un nodo cae, otro puede seguir dando el servicio.	

> Pregunta de examen real (1 punto): **«Ventajas de la computación distribuida»** → económica, compartición de recursos, escalabilidad y tolerancia a fallos. Si sobra tiempo, añade los 2 inconvenientes.

Formas de computación

Forma	Descripción	Ejemplo
Monolítica	Un único computador sin conexión y un solo usuario. (El *mainframe* deja varios usuarios conectados a un único ordenador, cada uno con su sesión.)	Un PC antiguo sin red
Distribuida	Muchos computadores conectados por red colaborando.	La web, el correo
Paralela	Más de un procesador a la vez para ejecutar un mismo programa. Es difícil dividir un programa en porciones que puedan ir a la vez.	Biología, aeronáutica, inteligencia artificial
Cooperativa	Dividir la computación entre ordenadores conectados para aprovechar los ciclos de CPU que les sobran.	Proyectos de cálculo voluntario

> Frase del profesor: **distribuida = muchos computadores**; **paralela = uso de varios procesadores**.

2 Comunicación entre procesos - IPC (Temas 1.2 y 2.1)

La idea en una frase

Si un sistema distribuido es un conjunto de procesos independientes, **la comunicación entre procesos (IPC)** es lo que los une: un proceso **emisor** envía datos y otro **receptor** los recibe. Para entenderse necesitan un **protocolo** (unas reglas): ¿quién habla primero?, ¿en qué formato?, ¿cómo se avisa de que se ha terminado?

- **Unicast**: la comunicación va de **un proceso a otro** (un cliente con un servidor).
- **Multicast**: de **un proceso a varios** a la vez.
- El **sistema operativo** ofrece mecanismos para comunicarse: **colas de mensajes, semáforos y regiones de memoria compartida**.
- Normalmente se programa sobre un **nivel de abstracción** que oculta el SO. En Java, ese nivel son las clases de **sockets** (capítulo 5).

Operaciones primitivas

Son las acciones básicas de cualquier comunicación. Cada proceso las invoca **de forma independiente** (el otro no sabe cuándo lo hará) y cada invocación provoca un **evento** (p. ej. *enviar* implica que se transmiten datos al receptor).

Operación	Quién la hace	Qué significa
Enviar	Emisor	Transmite datos hacia el receptor.
Recibir	Receptor	Recoge los datos que llegan.
Conectar	Ambos	Uno invoca *solicitar-conexión* y el otro *aceptar-conexión*.
Desconectar	Ambos	Libera los dos extremos de la comunicación.

> **Relación con Java**: `new Socket(..)` = solicitar conexión; `serverSocket.accept()` = aceptar conexión; `println()` = enviar; `readLine()` = recibir; `close()` = desconectar.

Sincronización de eventos: ¿se espera o no?

Los dos extremos son independientes, así que surge la pregunta: **¿cómo sabe uno que el otro ha terminado?** (Ejemplo: en HTTP, ¿cómo sabe el servidor que debe enviar los datos?). La forma más sencilla es usar **peticiones bloqueantes**: la operación **no termina** hasta que ocurre lo esperado.

Tipo	También se llama	Qué hace el programa	Ejemplo
Bloqueante	Síncrona	Se **queda esperando** hasta que la operación termina	La petición web: el navegador espera la respuesta
No bloqueante	Asíncrona	**Sigue ejecutando** sin esperar; se le avisa después	La respuesta del servidor llega «cuando llega»

> **Extra · analogía**: síncrono = una **llamada de teléfono** (hasta que no contestan no hablas con nadie más); asíncrono = una **carta** (la envías y sigues con tu vida; la respuesta llegará).

> **Bloqueo indefinido**: si lo que esperas **nunca llega**, una operación bloqueante nunca se desbloquea y el programa se queda colgado para siempre.

Las cuatro combinaciones de enviar y recibir

Enviar	Recibir	Qué ocurre (línea de tiempo)
Síncrono	**Síncrono**	El emisor empieza *enviar bloqueante* y el receptor *recibir bloqueante*. Cuando llegan los datos, la librería IPC devuelve un **asentimiento transparente** al emisor; ambos terminan.
Asíncrono	**Síncrono**	El emisor envía sin bloquearse y sigue. El receptor se bloquea hasta que llegan los datos; entonces su *recibir* retorna.
Síncrono	**Asíncrono**	**Escenario 1**: el receptor lanza *recibir no bloqueante* antes y cuando llegan los datos hay asentimiento y todo termina bien. **Escenario 2**: el receptor invoca *recibir no bloqueante* y **retorna inmediatamente** (aún no hay datos); luego el emisor invoca *enviar bloqueante* y no hay nadie esperando → **bloqueo indefinido**. **Escenario 3**: el receptor invoca recibir no bloqueante, retorna, y **se le notifica** la llegada de los datos cuando ocurre.
Asíncrono	**Asíncrono**	Nadie se bloquea. Al proceso receptor se le **notifica** la llegada de los datos y estos ya están disponibles.

```
SÍNCRONO + SÍNCRONO                    ASÍNCRONO (enviar) + SÍNCRONO (recibir)
Proceso 1            Proceso 2         Proceso 1             Proceso 2
enviar (espera) ---------------+       enviar (NO espera) -----------+
                  recibir (espera)                        recibir (espera)
   <---- asentimiento ----      |          sigue haciendo otras cosas   |
enviar retorna      recibir retorna                              recibir retorna
```

Cómo se evita el bloqueo indefinido

Dos soluciones, y es **pregunta clásica de examen**:

1. **Temporizadores (timers)**: se fija un **tiempo máximo de espera** para la operación. Si la respuesta no llega a tiempo, el proceso abandona la espera y puede reaccionar, en vez de quedarse colgado.
2. **Proceso hijo o hilo de ejecución**: la espera se hace en **otro hilo**, y el programa principal puede seguir haciendo otras tareas. Es útil para tareas lentas como **conexiones de red o grabación en disco**.

> **Extra · en Java**: un temporizador sobre un socket se programa con `socket.setSoTimeout(5000)` (si `readLine()` espera más de 5 s lanza `SocketTimeoutException`). La solución con hilos es todo el capítulo 4.

Comunicación con estado y sin estado

El **estado de un objeto** son los valores que tienen sus atributos en un momento dado (si un atributo vale 5, ese valor define su estado; mientras no cambie, el estado se mantiene).

Aplicado a la comunicación: si el proceso A envía información al proceso B y **B guarda información de las comunicaciones anteriores**, decimos que la conexión **tiene estado**. Si B no recuerda nada de peticiones anteriores, es **sin estado**.

> **Ejemplo del profesor**: ¿qué pasa cuando haces login en una web si HTTP no tiene estado? Que en cada página tendrías que volver a introducir tus datos, porque el servidor «no recuerda quién eres». **Solución: las cookies**: al iniciar sesión el servidor te envía un identificador (un número aleatorio); tu navegador lo guarda y lo manda en cada petición, y así el servidor te reconoce. *(Esto es poco importante en DAD I y más en DAD II.)*

3 Eclipse: crear, ejecutar y depurar proyectos

Qué vamos a hacer y cómo está organizado un proyecto

Todos los ejemplos de la asignatura (los proyectos *Hilos de ejecución* y *Sockets* del profesor) son **proyectos Java de Eclipse**. Antes de nada, conviene entender cómo se organizan:

Elemento	Qué es	Ejemplo
Workspace	La carpeta donde Eclipse guarda tus proyectos. La eliges al abrir Eclipse (por defecto `eclipse-workspace`).	`~/eclipse-workspace`
Proyecto	Una carpeta con todo el código de un trabajo.	`Sockets`, `Hilos de ejecucion`
Carpeta `src`	Aquí va el código fuente (`.java`).	`Sockets/src`
Paquete (package)	Una subcarpeta que agrupa clases relacionadas. Se escribe en **minúsculas** y con puntos; cada punto es una carpeta.	`edu.ucam.sockets.echo` → `src/edu/ucam/sockets/echo/`
Clase	Un fichero `.java`. **El nombre del fichero debe ser igual al nombre de la clase pública**, con la primera letra en mayúscula (*CamelCase*).	`Servidor.java` con `public class Servidor`

```
Sockets/                    <- proyecto
  src/                      <- código fuente
    edu/ucam/sockets/       <- paquetes (edu.ucam.sockets)
      primersocket/
        Servidor.java       <- package edu.ucam.sockets.primersocket;
        Cliente.java
      echo/
        Servidor.java       <- package edu.ucam.sockets.echo;
        Cliente.java
  JRE System Library        <- el Java que usa el proyecto
```

> La primera línea de cada clase, `package edu.ucam.sockets.echo;`, le dice a Java en qué paquete está. Eclipse la escribe sola cuando creas la clase desde el paquete; si mueves ficheros a mano y no coincide, aparece un error rojo (se arregla con **Ctrl+1 → Move to package**). El paquete se puede dejar vacío en pruebas pequeñas, pero en las prácticas del profesor se usa `edu.ucam.…`.

Paso a paso: crear un proyecto Java nuevo

1. Abre Eclipse. Si te pregunta el **workspace**, acepta el que viene por defecto y pulsa **Launch**. (Si aparece la pantalla de bienvenida, ciérrala con la X de la pestaña «Welcome».)
2. Menú **File → New → Java Project** (si no lo ves: **File → New → Other… → Java → Java Project**).
3. **Project name**: escribe el nombre, p. ej. `Examen` (sin tildes ni espacios si puedes evitarlo).
4. Deja **Use default location** marcado y, en **JRE**, **Use an execution environment JRE** (el JavaSE que traiga tu Eclipse, p. ej. JavaSE-17 o 21).
5. Si aparece la casilla **Create module-info.java file**, **desmárcala** (no la necesitamos; si al pulsar Finish te pregunta, responde **Don't Create**).
6. Pulsa **Finish**. En la vista de la izquierda (**Package Explorer**) verás el proyecto con su carpeta `src` y la *JRE System Library*.

> Si no ves el Package Explorer: **Window → Show View → Package Explorer**. Si la pantalla está desordenada: **Window → Perspective → Reset Perspective** (asegúrate de estar en la perspectiva **Java**).

Paso a paso: crear paquete y clase

1. En el Package Explorer, **clic derecho sobre** `src` **→ New → Package**. Escribe el nombre, p. ej. `edu.ucam.sockets.echo`, y **Finish**. (La casilla *Create package-info.java* déjala sin marcar.)
2. **Clic derecho sobre el paquete → New → Class**.
3. **Name**: `Servidor` (mayúscula inicial, sin `.java`). Modificador: **public**.
4. Si la clase va a **arrancar el programa** (tendrá `main`): marca **public static void main(String[] args)**.
5. Si es un **hilo**: en **Superclass** pulsa **Browse…** y elige `java.lang.Thread` (o `Thread` y Enter). Si implementa **Runnable**: en **Interfaces** pulsa **Add…** y elige `java.lang.Runnable`; con *Inherited abstract methods* marcado, Eclipse te crea el método `run()` vacío.
6. Pulsa **Finish**: se abre el editor con la clase creada y su línea `package …;`.

Repite para `Cliente`, `HiloServidor`, etc. **Una clase = un fichero.**

Escribir código con ayuda de Eclipse (atajos que ahorran tiempo)

En Mac usa **Cmd** donde ponga Ctrl; en Windows/Linux, Ctrl.

Atajo	Qué hace	Cuándo usarlo
`[Ctrl+Espacio]`	**Autocompletar** y sugerencias.	Escribe `main` + Ctrl+Espacio → genera todo el `public static void main`. Escribe `sysout` + Ctrl+Espacio → `System.out.println();`. También para ver los métodos de un objeto tras el punto.

Atajo	Qué hace	Cuándo usarlo
`[Ctrl+Shift+O]`	**Organizar imports**: añade los `import` que faltan y quita los sobrantes.	Cuando salga «X cannot be resolved to a type». Si hay varias opciones elige `java.io`…/`java.net`….
`[Ctrl+1]`	**Arreglo rápido** (*Quick Fix*) sobre el error o aviso del cursor.	«Unhandled exception type IOException» → *Surround with try/catch*. También para crear métodos o variables que faltan.
`[Ctrl+Shift+F]`	**Formatear** el código (indentación).	Antes de entregar, para dejarlo limpio.
`[Ctrl+S]`	Guardar.	Eclipse compila al guardar: los errores rojos aparecen al instante.
`[Ctrl+/]`	Comentar/descomentar la línea o selección.	Para anular `interrupt()` y probar variantes.
`[Alt+↑ / Alt+↓]`	Mover línea arriba/abajo.	
`[Ctrl+D]`	Borrar la línea actual.	
Source → Generate Constructor using Fields…	Crea el constructor a partir de los atributos.	Para `HiloServidor(Socket socket, Servidor servidor)`.
Source → Generate Getters and Setters…	Crea los `get`/`set`.	Para clases de datos (Empleado, Matricula…).
Source → Generate toString()…	Crea el `toString()`.	Para mostrar objetos como texto.
Source → Override/Implement Methods…	Sobrescribe métodos de la clase padre.	Para añadir `run()` si no lo generó.

> Las líneas rojas subrayadas son **errores de compilación** y las amarillas, **avisos** (los avisos como *Resource leak* no impiden ejecutar). Si pasas el ratón por encima, Eclipse te explica el problema y ofrece arreglos.

Paso a paso: ejecutar un programa y ver la salida

1. Con el fichero abierto en el editor (o seleccionado en el Package Explorer), **clic derecho → Run As → Java Application**. (También sirve el botón verde ▶ de la barra de herramientas o `[Ctrl+F11]`; si ejecuta otra clase distinta, usa mejor el clic derecho.)
2. Si pregunta **Save and Launch**, acepta (guarda y ejecuta).
3. La salida aparece en la vista **Console** (parte de abajo). Si no se ve: **Window → Show View → Console**.
4. Si el programa **pide datos por teclado** (`Scanner`), haz clic **dentro de la Console**, escribe y pulsa Enter.
5. Para parar un programa que sigue vivo: **cuadrado rojo ■ (Terminate)** de la Console, o `[Ctrl+F2]`. Cuando un servidor está en `while(true)` **nunca termina solo**: hay que pararlo tú.
6. Limpia la lista de ejecuciones acabadas con el icono de las dos X (**Remove All Terminated Launches**) y borra el texto con la escoba (**Clear Console**).

> Si ves `Error: Main method not found` o `Selection does not contain a main type` es que la clase que ejecutas **no tiene** `public static void main(String[] args)`, o es una clase de apoyo (p. ej. `HiloServidor`). Ejecuta la clase que sí tiene el `main`.

Paso a paso: probar un servidor y uno o varios clientes

Un programa de sockets son **dos programas** (servidor y cliente) que se ejecutan **a la vez** y se hablan. En Eclipse:

1. **Clic derecho sobre** `Servidor.java` **→ Run As → Java Application.** La consola muestra «Esperando cliente…» y se queda parada: está bloqueado en `accept()`. Es lo correcto.
2. **Clic derecho sobre** `Cliente.java` **→ Run As → Java Application.** Se abre una **segunda consola** (cada ejecución tiene la suya).

3. Para cambiar de consola usa el icono del monitor **Display Selected Console** (desplegable) en la barra de la Console. Cada ejecución aparece en la lista con su clase y la hora.
4. Escribe el comando **en la consola del cliente** y pulsa Enter; mira la respuesta en esa consola y los mensajes del servidor en la suya.
5. Para probar **varios clientes a la vez**: ejecuta `Cliente` otra vez (Run As de nuevo). Tendrás una consola por cliente.
6. Al terminar, **para el servidor con ■**. Si lo dejas vivo y vuelves a lanzarlo, saldrá `BindException: Address already in use` porque el puerto sigue ocupado.

> **Orden**: primero el **Servidor**, después el **Cliente**. Si arrancas antes el cliente: `ConnectException: Connection refused`.

> **En macOS el puerto 5000 lo usa AirPlay** (Control Center): un `new ServerSocket(5000)` dará `BindException`. Cambia a un puerto libre (`5050`, `8000`…) **en servidor y cliente**. En el aula (Windows) el 5000 funciona.

Importar los proyectos del profesor (los .zip) y entregar el tuyo

Los ficheros `DAD1 - hilos.zip` y `DAD1 - sockets.zip` son proyectos Eclipse exportados. Para abrirlos:

1. **File → Import…** → carpeta **General** → **Existing Projects into Workspace** → **Next**.
2. Marca **Select archive file** → **Browse…** → elige el `.zip`.
3. Debe aparecer el proyecto marcado en la lista → **Finish**. Ya está en tu Package Explorer; abre sus paquetes y ejecuta las clases.

Para exportar el tuyo como zip: **File → Export… → General → Archive File** → marca el proyecto → elige destino y **Finish**.

> Si al importar falla porque el JRE no existe: clic derecho en el proyecto → **Properties → Java Build Path → Libraries** → quita la JRE en rojo → **Add Library… → JRE System Library** → elige la que tengas.

Interfaz gráfica de los talleres (Swing con WindowBuilder)

El taller de hilos usa una ventana con botones. Se crea con el plugin **WindowBuilder** de Eclipse:

1. **File → New → Other… → WindowBuilder → Swing Designer → Application Window**. Ponle nombre (p. ej. `Hilo`) y Finish.
2. Abajo del editor hay dos pestañas: **Source** (código) y **Design** (diseño visual con paleta de componentes y panel de propiedades).
3. En **Design** arrastra componentes desde la paleta: `JButton` (botón), `JTextArea` (área de texto, mejor dentro de un `JScrollPane`) y `JComboBox` (lista desplegable).
4. Para darle una acción a un botón: **doble clic sobre el botón** (genera un `actionPerformed`) o crea una clase aparte que implemente `ActionListener` y asígnala con `boton.addActionListener(new ActionLanzarHilo(this))` (así lo hizo el profesor).
5. Ejecútala con **Run As → Java Application** (la clase generada trae su propio `main`).

> Si no existe **WindowBuilder** en el menú: **Help → Eclipse Marketplace…**, busca «WindowBuilder» e instálalo. Para el examen de programación **no se usa interfaz gráfica**: solo clases con `main` y sockets.

Errores típicos de Eclipse y cómo arreglarlos

Mensaje / síntoma	Causa	Solución
`X cannot be resolved to a type`	Falta un `import` o está mal escrito el nombre.	`[Ctrl+Shift+O]` o escribir el import.
`Unhandled exception type IOException`	Un método que puede lanzar `IOException` (sockets, readLine) no está dentro de `try/catch`.	`[Ctrl+1]` → *Surround with try/catch*.
`The public type X must be defined in its own file`	El nombre de la clase pública no coincide con el del fichero.	Renombra la clase o el fichero (clic derecho → *Refactor* → *Rename*).
`The declared package … does not match the expected package`	La línea `package` no corresponde a la carpeta.	`[Ctrl+1]` sobre el error → mover o cambiar el paquete.
`Selection does not contain a main type`	Ejecutas una clase sin `main`.	Ejecuta la clase con `public static void main`.
`java.net.BindException: Address already in use`	Otro programa (o tu servidor anterior) usa el puerto.	Para el anterior con ■ en la consola; o cambia de puerto. En Mac, evita el 5000.
`java.net.ConnectException: Connection refused`	El cliente no encuentra servidor en esa IP/puerto.	Arranca antes el servidor y comprueba que el puerto coincide.
El programa «se queda colgado» y no escribe nada	Está bloqueado en `accept()` o `readLine()` esperando; o falta `flush()`.	Es normal en un servidor que espera. Si es el cliente, revisa `flush()` y que haya servidor.
Cambias el código y no se nota	Sigue corriendo la ejecución antigua.	Para (■) y vuelve a ejecutar.
Muchas consolas abiertas	Cada ejecución crea una.	*Remove All Terminated Launches* y *Display Selected Console*.

Plan de trabajo en el examen con Eclipse

1. Crea el proyecto (**File** → **New** → **Java Project**) y un paquete (o usa el paquete por defecto).
2. Crea **Servidor** (con `main`), **HiloGestionXxx** (extends `Thread`) y, si hace falta, la clase de datos (`Empleado`, `Matricula`…). Si te piden **solo el hilo**, escribe solo esa clase pero con todos sus imports.
3. Escribe primero el **esqueleto** (atributos, constructor, `run()` con buffers y bucle). Guarda con `[Ctrl+S]` y mira que no haya rojos.
4. Añade los comandos uno a uno dentro del `switch`. Pulsa `[Ctrl+Shift+O]` y `[Ctrl+Shift+F]` al terminar.
5. Si puedes, **pruébalo**: ejecuta el Servidor y un Cliente y lanza los comandos del enunciado. Termina siempre el servidor (■) al acabar.

4 Hilos de ejecución en Java

Qué es un hilo (con analogía)

> **Hilo de ejecución**: la **secuencia única de control de flujo** dentro de un programa; la **unidad de código más pequeña que se puede ejecutar**.

Todo programa Java arranca con **un hilo**: el hilo `main`, que ejecuta las instrucciones una detrás de otra. Un **hilo adicional** es otra «línea de ejecución» que corre **a la vez** que el main dentro del mismo programa.

> **Extra · analogía**: una cocina. Si cocinas tú solo (un hilo) haces los pasos uno tras otro: cortas, fríes, sirves. Con ayudantes (varios hilos) uno corta mientras otro fríe *al mismo tiempo*, pero comparten la misma cocina (la memoria del programa), así que deben coordinarse para no pisarse (ver «sincronización»).

Ventaja	Qué significa	Ejemplo
Ejecución de tareas lentas	Lo lento se hace en segundo plano.	Descargar un fichero, grabar en disco
No paralizar el programa principal	Mientras un hilo trabaja, el principal sigue respondiendo.	La ventana no se congela
Evitar bloqueos inaceptables	Un bloqueo indefinido de una parte no cuelga todo el programa.	Un cliente lento no bloquea a los demás
Manejo de múltiples tareas	Un proceso se divide para hacer varias cosas a la vez.	Un servidor con varios clientes

Crear un hilo: dos formas

En las dos formas el trabajo que hará el hilo se escribe **dentro del método** `run()`. La diferencia es cómo se construye y se lanza:

	1) Heredar de `Thread`	2) Implementar `Runnable`
Declaración	`public class MiHilo extends Thread`	`public class MiHilo implements Runnable`
Método obligatorio	`public void run()`	`public void run()`
Lanzar	`MiHilo h = new MiHilo(); h.start();`	`Thread t = new Thread(new MiHilo()); t.start();`
Ventaja	Más corto.	La clase todavía puede heredar de otra (Java no permite herencia múltiple).

Ejemplo 1: heredando de Thread (proyecto *Hilos de ejecución*, paquete `primerhilo`)

```
package edu.ucam.hilos.primerhilo;

import java.util.Iterator;

public class MiHilo extends Thread{

    @Override
    public void run() {
        for (int i = 0 ; i< 5 ; i++) {
            System.out.println("Ejecución: " + i);

            try {
                Thread.sleep(500);
            } catch (InterruptedException e) {
                e.printStackTrace();
            }

        }
    }

}
```

```
package edu.ucam.hilos.primerhilo;

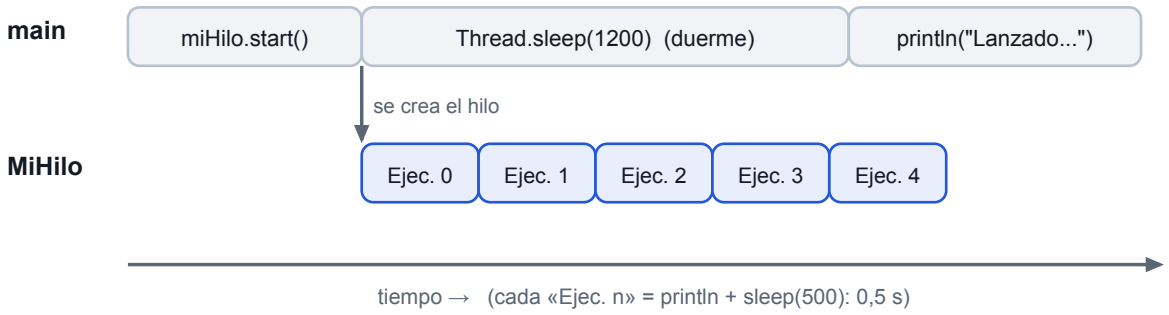
public class Root {

    public static void main(String[] args) {
        MiHilo miHilo = new MiHilo();
        miHilo.start();

        try {
            Thread.sleep(1200);
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
        System.out.println("Lanzado...");
    }
}
```

Explicación línea a línea:

- `public class MiHilo extends Thread` → «MiHilo *es un* hilo»: hereda todo lo de `Thread`.
- `@Override public void run()` → aquí está **lo que hará el hilo**. `@Override` indica que sustituimos el `run()` de `Thread`.
- El `for` imprime «Ejecución: 0…4» y entre cada vuelta `Thread.sleep(500)` **duerme** el hilo medio segundo. `sleep` obliga a capturar `InterruptedException` (por si alguien interrumpe el sueño).
- En el `Root` (clase con `main`): `new MiHilo()` crea el objeto; `start()` **lanza el hilo** y el `main` sigue por su cuenta *sin esperar*.
- El main duerme 1,2 s y escribe «Lanzado...». Mientras, el hilo ya está escribiendo.



El hilo secundario arranca con `start()` y avanza **en paralelo** al main: el «Lanzado...» sale **en medio** de las ejecuciones (a 1,2 s), no al final.

```
Salida:
Ejecución: 0      (t = 0 s)
Ejecución: 1      (t = 0,5 s)
Ejecución: 2      (t = 1,0 s)
Lanzado...        (t = 1,2 s)   <- el main, en paralelo
Ejecución: 3      (t = 1,5 s)
Ejecución: 4      (t = 2,0 s)
```

Ejemplo 2: implementando Runnable (paquete `primerhilointerfaz`). El hilo es idéntico; lo que cambia es el lanzamiento:

```
package edu.ucam.hilos.primerhilointerfaz;

public class MiHilo implements Runnable{

    @Override
    public void run() {
        for (int i = 0 ; i< 5 ; i++) {
            System.out.println("Ejecución: " + i);

            try {
                Thread.sleep(500);
            } catch (InterruptedException e) {
                e.printStackTrace();
            }
```

```
        }
    }

}
```

```
package edu.ucam.hilos.primerhilointerfaz;

public class Root {

    public static void main(String[] args) {
        MiHilo miHilo = new MiHilo();

        Thread thread = new Thread(miHilo);
        thread.start();
        try {
            Thread.sleep(1200);
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
        System.out.println("Lanzado...");
    }
}
```

- `MiHilo` ya **no** es un `Thread`, solo **un trabajo** (`Runnable`) que sabe hacer `run()`. Por eso **no tiene** `start()`.
- Para ejecutarlo hay que **entregárselo a un** `Thread`: `new Thread(miHilo)`, y es ese `Thread` el que se lanza con `start()`.

> **«No puedes crear objetos de interfaces»** (apuntes del profesor): `Runnable` es una **interfaz**, así que `new Runnable()` a secas no vale; se crea un objeto de *tu clase* que la implementa (`new MiHilo()`).

start() frente a run(): el error más preguntado

	`start()`	`run()`
¿Crea un hilo nuevo?	**Sí**	**No**
¿Dónde se ejecuta el código?	En el hilo nuevo (`Thread-0`…)	En el hilo que lo llama (normalmente `main`)
¿Puede llamarse varias veces sobre el mismo objeto?	**No**: `IllegalThreadStateException`	Sí, pero es un método normal

```
Thread t = new Thread(new Tarea());
t.run();      // ¡MAL! se ejecuta en main, secuencialmente, no hay hilo nuevo
t.start();    // BIEN: crea el hilo y ejecuta run() en él

MiHilo m = new MiHilo();
m.start();    // OK
m.start();    // IllegalThreadStateException: un hilo solo se arranca una vez
```

> Para lanzar **10 hilos** crea **10 objetos**: `for (int i = 0; i < 10; i++) { new MiHilo().start(); }`. Con `Runnable`: `new Thread(miRunnable).start()` (el mismo Runnable puede compartirse, pero cada `Thread` es nuevo).

Ciclo de vida de un hilo (Extra)

Estado (Java)	Cuándo
`NEW`	Creado con `new` pero aún sin `start()`.
`RUNNABLE`	Ejecutándose o listo para ejecutarse.
`TIMED_WAITING`	Dormido con `sleep(ms)` o esperando con `join(ms)`.
`BLOCKED / WAITING`	Esperando un `synchronized` o una espera sin límite.
`TERMINATED`	Su `run()` ha acabado. **No se puede reiniciar.**

Por eso `start()` solo vale una vez: un hilo `TERMINATED` (o ya arrancado) no vuelve a `NEW`.

Esperar y parar hilos: sleep, join e interrupt

Método	Qué hace
`Thread.sleep(ms)`	**Duerme el hilo actual** ms milisegundos. Lanza `InterruptedException`.
`hilo.join()`	El hilo que lo llama **espera a que** `hilo` **termine**.
`hilo.join(ms)`	Espera como **máximo** ms milisegundos (es una espera con temporizador).
`hilo.interrupt()`	**Interrumpe** al hilo: si está en `sleep` o `join`, salta `InterruptedException` dentro de él.

Ejemplo 3: `pararhilo`. Un hilo duerme 10 s; el main lo espera como máximo 5 s:

```
package edu.ucam.hilos.pararhilo;

public class UnHilo extends Thread {

    public void run() {
        try {

            System.out.println("[Hilo secundario]: Estoy ejecutándome...");

            Thread.sleep(10000);

            System.out.println("[Hilo secundario]: Finalizo hilo");

        } catch (InterruptedException e) {
            System.out.println("Excepción en hilo: se ha interrumpido");
        }

        }

}
```

```
package edu.ucam.hilos.pararhilo;

public class Root {

    public void ejecutar() {
        try {

            UnHilo unHilo = new UnHilo();

            unHilo.start();

            System.out.println("[Main]: Me paro antes del join");
            unHilo.join(5000);
            //unHilo.interrupt();

            System.out.println("[Main]: Fin en Root");

        } catch (InterruptedException e) {

            e.printStackTrace();
        }

    }

    public static void main(String[] args) {
        (new Root()).ejecutar();
    }
}
```

```
Sin interrupt():
[Main]: Me paro antes del join          (t = 0 s)
[Hilo secundario]: Estoy ejecutándome... (t = 0 s)
[Main]: Fin en Root                      (t = 5 s)   <- el join se acabó por tiempo
[Hilo secundario]: Finalizo hilo         (t = 10 s)  <- el hilo SIGUE vivo y termina solo

Con unHilo.interrupt() descomentado tras el join:
... Fin en Root (t = 5 s) y a continuación: "Excepción en hilo: se ha interrumpido" (el sleep se corta)
```

> **Ejemplo del taller (Swing)**: para parar un hilo con un botón se usa un atributo `boolean running` que el bucle del hilo comprueba (`while (running) {…}`) y un método `parar()` que lo pone a `false`. Se declara `volatile` para que el hilo vea el cambio hecho desde otro hilo.

Compartir datos: condición de carrera y synchronized

Los hilos de un mismo programa **comparten la memoria**. Si varios **modifican la misma variable** a la vez, pueden pisarse. Ejemplo típico: `contador++` parece una sola operación pero son **tres**: **leer** el valor, **sumar 1** y **escribir** el resultado.

Paso	Hilo A	Hilo B	Valor en memoria
1	lee 5		5
2		lee 5 (A aún no ha escrito)	5
3	calcula 6 y escribe 6		6
4		calcula 6 y escribe 6	**6** (debería ser 7: se perdió un incremento)

A esto se le llama **condición de carrera**: el resultado depende del orden en que se intercalen los hilos. Por eso 5 hilos sumando 1000 veces **no dan 5000 sino un número menor y distinto en cada ejecución**.

Solución: `synchronized`. Un método o bloque `synchronized` solo puede ser ejecutado por **un hilo a la vez** sobre el mismo objeto; los demás esperan su turno.

```
package edu.ucam.hilos.contador;

//Nombre de clase debe ser Camel Sense. No válidos: contador o cONTADOR...
public class Contador {

    private int total = 0;

    public synchronized void sumaUno() {
        this.total = this.total + 1;
    }

    public int getTotal() {
        return this.total;
    }
}
```

```
package edu.ucam.hilos.contador;

public class HiloContador extends Thread {

    private Contador contador;

    public HiloContador (Contador contador) {
        this.contador = contador;
    }

    public Contador getContador() {
        return contador;
    }
    public void setContador(Contador contador) {
        this.contador = contador;
    }

    public void run() {

        for(int i = 0 ; i < 100; i++) {
            this.contador.sumaUno();
        }

        try {
            Thread.sleep(200);

            System.out.println("Total contador: " + this.contador.getTotal());
```

```
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
    }
}
```

```
package edu.ucam.hilos.contador;

public class Root {

    public void ejecutar() {

        Contador contador = new Contador();

        for (int i = 0; i < 200; i++) {
            (new HiloContador(contador)).start();
        }
    }

    public static void main(String[] args) {
        Root root = new Root();
        root.ejecutar();
    }
}
```

- `Contador` guarda `total` y su método `sumaUno()` es `synchronized`: el incremento ya es atómico.
- `Root` lanza **200 hilos** (200 objetos `HiloContador`), todos con **el mismo** `Contador`.
- Cada hilo suma 100 veces, duerme 200 ms (para dar tiempo a que acaben los demás) e imprime el total: el resultado final correcto es **200 × 100 = 20 000**. Sin `synchronized` saldrían valores menores.

Alternativas: `AtomicInteger` (`incrementAndGet()`) o un bloque `synchronized (objeto) { … }`. **Importante**: la causa del fallo es **compartir el mismo objeto** (`new Thread(hc)` cinco veces con el mismo `hc`); si cada hilo tuviera su propio contador no habría carrera.

Regla del examen: «¿por qué el contador no llega a 5000?» → los hilos comparten el mismo objeto/variable y `contador++` no es atómico (condición de carrera). Se corrige con **synchronized**.

Los cuatro errores de hilos que más caen

Código	Qué pasa	Por qué / arreglo
`for (...) { (new Thread()).start(); }` con `MiHilo implements Runnable`	No se imprime el texto del hilo.	El `Thread` se crea **sin pasarle el Runnable**: su `run()` está vacío. → `new Thread(miHilo).start()`.
`for (...) { miHilo.start(); }` con un único objeto	`IllegalThreadStateException`.	Un hilo solo se arranca **una vez**. → crear un objeto nuevo en cada vuelta.
5 hilos × `for(1000){ contador++ }` sobre el mismo objeto	Resultado < 5000 y variable.	Condición de carrera. → `synchronized`/`AtomicInteger`.
`t.run();`	Todo ocurre en `main`, sin concurrencia.	`run()` es un método normal. → `t.start()`.

5 Sockets (Tema 2.2)

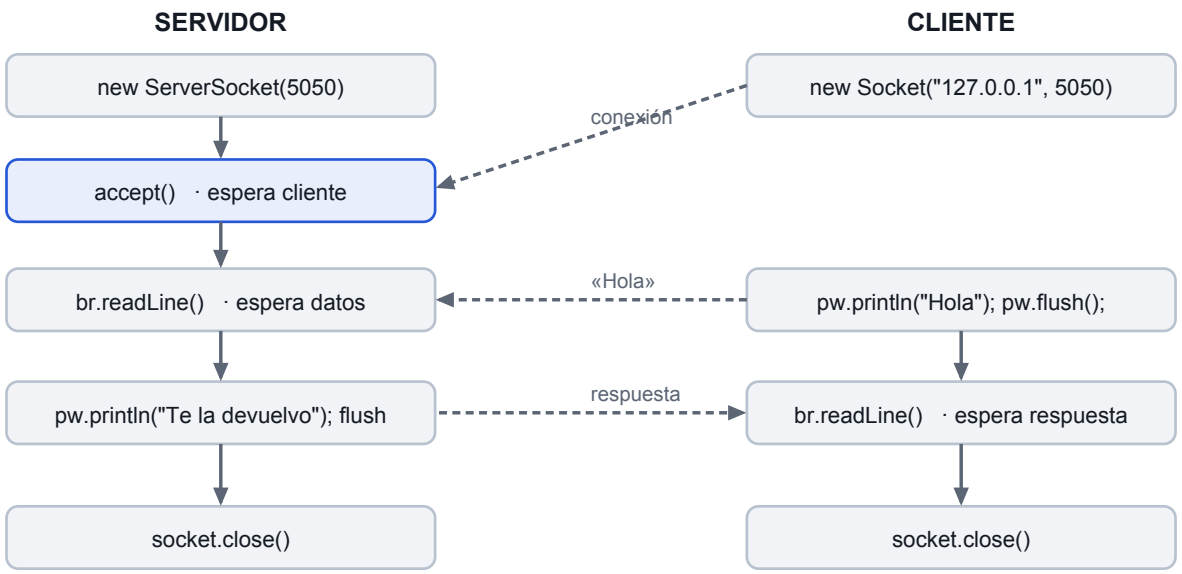
Qué es un socket

> **Socket**: **punto de conexión** entre dos procesos, identificado por una **IP y un puerto**.

- **IP**: la dirección de la máquina en la red (`127.0.0.1` o `localhost` = **esta misma máquina**, ideal para pruebas).
- **Puerto**: un número (0–65535) que identifica **qué programa** de esa máquina recibe los datos. Un servidor elige su puerto y los clientes lo usan para llegar a él.
- En la comunicación hay **dos sockets**, uno en cada extremo; **los dos puertos (el del servidor y el del cliente) identifican la comunicación**. El del servidor lo elegimos nosotros; el del cliente lo asigna el sistema operativo al azar.
- Para comunicarte necesitas **un objeto de la clase** `Socket` (igual que para tener un hilo necesitas la clase `Thread`). Y, como siempre, **tiene que haber un protocolo**.

Cómo se establece una conexión: servidor y cliente

Paso	Servidor	Cliente
1	**Crear socket** (`socket()`)	**Crear socket** (`socket()`)
2	`bind()`: asocia el socket a un puerto del sistema operativo (le dice al SO «este proceso ocupa este puerto»). Nosotros especificamos el puerto del servidor.	`bind()`: se le asigna un puerto (aleatorio, lo genera el programa/SO).
3	`listen()`: permite que se puedan conectar a nuestra máquina.	
4	`accept()`: **acepta la petición entrante** (y se queda esperando hasta que llega).	Se conecta al servidor (IP + puerto).
5	`leer()` / `escribir()`	`escribir()` / `leer()` (el que lee espera a que el otro escriba)
6	**Cerrar**	**Cerrar**



· = la ejecución se detiene (llamada bloqueante). Flechas discontinuas = datos por la red.

Mientras el servidor está en `accept()` o `readLine()`, **ese hilo no avanza**: está esperando. Por eso un servidor con un solo hilo atiende a un único cliente a la vez.

En Java, `new ServerSocket(puerto)` hace por ti socket + bind + listen. **Pregunta de examen: «¿cómo se activa un socket (servidor)?» → con** `accept()`: el servidor se para esperando y continúa cuando el cliente se conecta.

Las dos clases de Java y los flujos de lectura/escritura

Clase	Se usa en	Constructor / métodos clave
`ServerSocket`	Servidor	`new ServerSocket(int puerto)` · `accept()` (espera y **devuelve un** `Socket` conectado a ese cliente) · `close()`
`Socket`	Cliente (y, en el servidor, el que devuelve `accept()`)	`new Socket(String host, int puerto)` · `getInputStream()` · `getOutputStream()` · `close()`

Un socket te da **flujos de bytes**: `getInputStream()` (lo que **llega**) y `getOutputStream()` (lo que **sale**). Para trabajar cómodamente con **texto línea a línea** se «envuelven» en otras clases, como cajas dentro de cajas:

```
LEER:    socket.getInputStream()  ->  InputStreamReader  ->  BufferedReader
         (bytes que llegan)           (bytes -> caracteres)  (permite readLine(): una línea entera)

ESCRIBIR: PrintWriter  ->  OutputStreamWriter  ->  socket.getOutputStream()
          (println)        (caracteres -> bytes)    (bytes que salen)
```

```
BufferedReader br = new BufferedReader(new InputStreamReader(socket.getInputStream()));
PrintWriter    pw = new PrintWriter(new OutputStreamWriter(socket.getOutputStream()));

String linea = br.readLine();    // BLOQUEA hasta que llega una línea; devuelve null si el otro cerró
pw.println("texto a enviar");    // escribe la línea (con salto de línea al final)
pw.flush();                      // ¡IMPRESCINDIBLE! fuerza el envío; sin flush se queda en el buffer
```

Tres detalles que cuestan puntos: (1) sin `flush()` el otro lado se queda esperando para siempre; (2) `readLine()` devuelve `null` cuando el otro extremo cierra la conexión; (3) el protocolo es **una línea por mensaje**: `println` envía y `readLine` lee exactamente una línea.

Orden importante en el servidor (apuntes): primero `new ServerSocket(PUERTO)`, después `accept()` (que devuelve el `Socket`) y solo entonces crear los buffers sobre ese socket. En el cliente, primero `new Socket(host, PUERTO)` y luego los buffers.

Ejemplo 1 (primersocket): el cliente dice «Hola!!» y el servidor responde

Servidor (se ejecuta primero):

```
package edu.ucam.sockets.primersocket;

import java.io.BufferedReader;
import java.io.IOException;
import java.io.InputStreamReader;
import java.io.OutputStreamWriter;
import java.io.PrintWriter;
import java.net.ServerSocket;
import java.net.Socket;

public class Servidor {

    public void ejecutar() {
        try {
            ServerSocket serverSocket = new ServerSocket(5000);

            System.out.println("Servidor lanzado esperando cliente...");

            Socket socket = serverSocket.accept();

            PrintWriter pw = new PrintWriter(new OutputStreamWriter(socket.getOutputStream()));
            BufferedReader br = new BufferedReader(new InputStreamReader(socket.getInputStream()));

            String linea = br.readLine();

            System.out.println("Linea recibida en servidor: " + linea);
            pw.println("Te la devuelvo " + linea);
            pw.flush();

        } catch (IOException e) {
            // TODO Auto-generated catch block
            e.printStackTrace();
        }

    }

    public static void main(String[] args) {
        (new Servidor()).ejecutar();

    }
}
```

Cliente:

```
package edu.ucam.sockets.primersocket;

import java.io.BufferedReader;
import java.io.IOException;
import java.io.InputStreamReader;
import java.io.OutputStreamWriter;
import java.io.PrintWriter;
import java.net.Socket;
import java.net.UnknownHostException;

public class Cliente {
    public void ejecutar() {
        Socket socket;
        try {
            socket = new Socket("127.0.0.1",5000);

            PrintWriter pw = new PrintWriter(new OutputStreamWriter(socket.getOutputStream()));
            BufferedReader br = new BufferedReader(new InputStreamReader(socket.getInputStream()));

            pw.println("Hola!!"); pw.flush();

            System.out.println(br.readLine());

        } catch (UnknownHostException e) {
            e.printStackTrace();
        } catch (IOException e) {
            e.printStackTrace();
        }
```

```
    }

    public static void main(String[] args) {
        (new Cliente()).ejecutar();

    }

}
```

1. El servidor crea el `ServerSocket(5000)` y escribe «Servidor lanzado esperando cliente…». **accept()** lo bloquea.
2. El cliente hace `new Socket("127.0.0.1", 5000)`: se conecta; `accept()` del servidor se desbloquea y devuelve el socket de ese cliente.
3. El cliente escribe `"Hola!!"` (`println` + `flush`) y espera con `readLine()`.
4. El servidor lee la línea con `readLine()`, la imprime y responde `"Te la devuelvo Hola!!"`. El cliente la recibe e imprime.
5. Al acabar `ejecutar()`, ambos programas terminan.

El puerto del cliente (`new Socket(.., 5000)`) **debe ser el mismo** que el del `ServerSocket`. Si no, `Connection refused`. **En macOS cambia 5000 por 5050** en las dos clases.

Ejemplo 2 (echo): lo que escribes en el cliente te lo devuelve el servidor

El cliente lee del **teclado** con `Scanner` en un bucle, envía cada línea y muestra la respuesta; el servidor la imprime y la devuelve. **Servidor** y **cliente** (apartados esenciales):

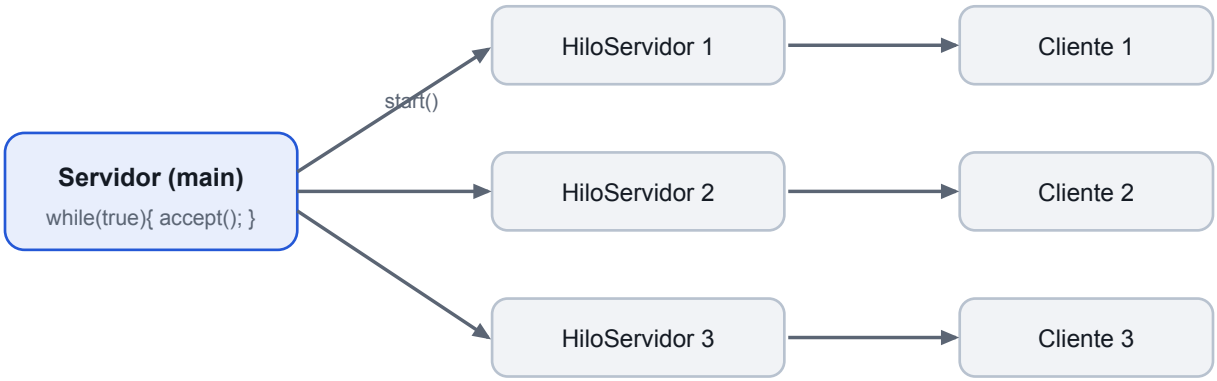
```
// SERVIDOR (echo)
Socket socket = serverSocket.accept();
while (true) {
    PrintWriter pw = new PrintWriter(new OutputStreamWriter(socket.getOutputStream()));
    BufferedReader br = new BufferedReader(new InputStreamReader(socket.getInputStream()));
    String linea = br.readLine();
    System.out.println("Linea recibida en servidor: " + linea);
    pw.println(linea);  pw.flush();                // eco
}

// CLIENTE (echo)
Scanner teclado = new Scanner(System.in);          // lee de la consola
while (true) {
    String linea = teclado.nextLine();
    pw.println(linea);  pw.flush();
    System.out.println(br.readLine());
}
```

Este servidor atiende **a un solo cliente**: un segundo cliente conecta pero nadie le responde, porque el servidor sigue dentro de su bucle con el primero. Se resuelve con un hilo por cliente (siguiente apartado). Con el cliente cerrado, `readLine()` da `null` y acaba saltando una excepción: «Conexión finalizada».

Servidor multihilo: un hilo por cliente

Ejercicio del profesor: *«cliente no tocar; servidor hay que bifurcar: objeto Thread (clase) y run; leer y hablar con cliente; si llega un cliente lanza un nuevo hilo»*.



El hilo principal solo espera conexiones; cada cliente tiene **su propio hilo** con **su propio socket**, y todos funcionan a la vez.

- El **hilo principal** del servidor solo hace **bucle de** `accept()`: por cada cliente que llega crea un `HiloServidor` pasándole el `Socket` y lo lanza con `start()`; vuelve a esperar al siguiente cliente **inmediatamente**.
- `HiloServidor extends Thread` tiene un atributo `Socket` que recibe en el **constructor** («permite a este hilo manejar la comunicación de manera exclusiva con ese cliente»).
- En su `run()` va **todo lo que antes estaba en el servidor**: crear los buffers, leer, responder, cerrar. Así las conversaciones son **concurrentes** y no bloquean al servidor ni a otros clientes.

```
// Servidor.java
while (true) {
    Socket socket = serverSocket.accept();
    new HiloServidor(socket).start();
}
```

```
// HiloServidor.java
public class HiloServidor extends Thread {
    private Socket socket;
    public HiloServidor(Socket socket) { this.socket = socket; }
    @Override public void run() { /* leer, responder, cerrar */ }
}
```

Regla: si el enunciado habla de «varios clientes» o «clientes simultáneos», la respuesta es **un hilo por cliente**. El estado compartido (p. ej. la lista de empleados) vive en el **servidor** (o en un atributo `static`) y se accede con `synchronized`.

Cerrar bien: try-with-resources y fin del bucle

```
try (Socket socket = new Socket("127.0.0.1", 5050);
     BufferedReader br = ...; PrintWriter pw = ...) {
    ...
} catch (IOException e) {
    e.printStackTrace();
}
```

```
// recursos entre paréntesis:
// se cierran SOLOS al acabar el try
```

Esta forma (**try con recursos**) la usó el profesor en uno de sus ejemplos y evita olvidar el `close()`. En un servidor con `while(true)` hay que salir del bucle cuando `readLine()` devuelva `null` o llegue `EXIT`, para que el hilo **termine** y se cierre el socket.

6 Protocolos de comunicación: HTTP y FTP (Tema 2.3)

Tipos de protocolos

Un **protocolo** es el conjunto de reglas que acuerdan dos programas para entenderse. Los dos de este tema comparten dos rasgos:

- **Basados en texto**: se envían **cadenas de caracteres** (representación tipo ASCII), legibles por una persona.
- **De solicitud-respuesta**: un lado invoca una **petición** y espera una **respuesta**.

> **Conexión con el examen**: los comandos del servidor de fichaje (`ALTAEMPLEADO 1234 Juan` → `Ok. Empleado insertado`) son **exactamente un protocolo de texto de solicitud-respuesta** inventado por el enunciado.

HTTP: qué es y características

> **HTTP** (HyperText Transfer Protocol, **RFC 1945**): protocolo de comunicación que **permite la transferencia de información en la web**; facilita la comunicación entre **navegadores y servidores**. **No es lo mismo que HTML** (HTML es el lenguaje de las páginas; HTTP es cómo viajan).

Característica	Explicación
Basado en **URL**	Una URL (Universal Resource Locator) identifica un **objeto al que se le puede pedir la ejecución de un método** y obtener un resultado según los parámetros.
Petición de acción	Un cliente pide una acción al servidor y éste le envía el resultado de realizarla.
Versiones	**0.9** (texto sin formato) → **1.0** [RFC 1945] (incluye formatos MIME) → **1.1** (nuevos servicios, reinicio de comunicaciones [RFC 2616]).
Muy extendido	Lo usa el **100 %** de usuarios y servidores.
Genérico	Protocolo tan genérico que **permite definir tus propios comportamientos**.
Sin estado	**No recuerda peticiones de clientes**. Primera solución: **cookies**.

> **Extra · partes de una URL**: `http://www.w3.org:80/pub/WWW/TheProject.html?x=1` = *protocolo* `http` + *dominio* `www.w3.org` + *puerto* `80` (por defecto, se omite) + *ruta* `/pub/WWW/TheProject.html` + *parámetros* `?x=1`. El profe: «identificamos un dominio después de www y el resto es mucho texto».

HTTP: estructura de un mensaje

Hay **dos tipos de mensajes**: **petición (request)** y **respuesta (response)**. Los dos tienen la misma forma:

Parte	Contenido
1. Primera línea	Distinta en petición y en respuesta (ver tabla siguiente).
2. Cabeceras (headers)	Metainformación en pares `nombre-del-campo:valor` acabados en CRLF. Hay cabeceras generales, de petición/respuesta y de entidad. *Una de ellas indica el tamaño del cuerpo.*
3. Línea en blanco	Un **CRLF** (retorno de carro + salto de línea) que separa las cabeceras del cuerpo.
4. Cuerpo (entity body)	La información: en la petición, datos o parámetros para el método; en la respuesta, el contenido pedido (p. ej. la página).

	Primera línea	Ejemplo
Petición	**Método** (GET, POST, HEAD…) + **recurso elegido (URL)** + **versión de HTTP**	`GET /pub/WWW/TheProject.html HTTP/1.0`
Respuesta	**Versión de HTTP** + **código de estado** (3 dígitos) + **frase descriptiva**	`HTTP/1.0 404 Not Found`

```
PETICIÓN (cliente -> servidor)                RESPUESTA (servidor -> cliente)
GET /index.html HTTP/1.0        <- 1ª línea     HTTP/1.0 200 OK                <- 1ª línea (estado)
Host: www.ejemplo.com           <- cabeceras    Content-Type: text/html        <- cabeceras
Accept: text/html                               Content-Length: 120
                                <- línea en blanco                             <- línea en blanco
(sin cuerpo en un GET)                          <html> ... </html>             <- cuerpo
```

> Pregunta del parcial (1,5 puntos): **«Características del protocolo HTTP. ¿Cuál es la estructura de un paquete HTTP?»** → las características de la tabla de arriba + **primera línea, cabeceras, línea en blanco CRLF y cuerpo**, distinguiendo petición y respuesta.

HTTP: métodos y códigos de estado

Método	Función	Versión
GET	**Recupera** la URL especificada.	1.0
POST	**Envía datos** a la URL especificada.	1.0
HEAD	Igual que GET pero solo devuelve las cabeceras.	1.0
PUT	**Guarda** los datos en la URL.	1.1
PATCH	Como PUT pero con una lista de diferencias.	1.1
COPY / MOVE	Copia / mueve el recurso a otras direcciones.	1.1
DELETE	**Borra** el recurso especificado.	1.1
LINK / UNLINK	Establece / quita ligaduras entre la URL y otras.	1.1
TRACE	Responde en el cuerpo la petición del cliente.	1.1
OPTIONS	Pide información sobre las opciones de comunicación de la URL.	1.1
WRAPPED	Permite agrupar y encriptar peticiones para más seguridad.	1.1

> El profesor: «no hay que aprender todos los métodos». Los 4 importantes: **GET** (recupera), **POST** (envía), **PUT** (guarda) y **DELETE** (borra).

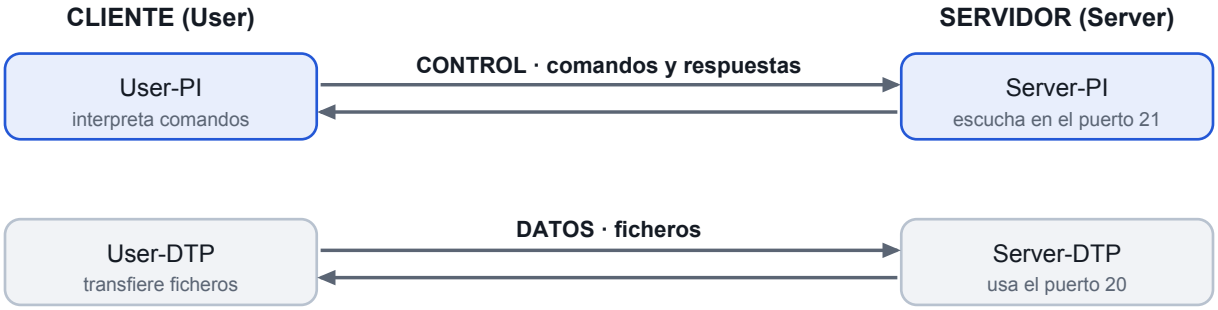
Código	Familia (Extra)	Ejemplos habituales
1xx	Informativo	100 Continue
2xx	Éxito	200 OK
3xx	Redirección	301 Moved Permanently
4xx	Error del cliente	404 Not Found (recurso inexistente)
5xx	Error del servidor	500 Internal Server Error

FTP: qué es y qué pretende

> **FTP** (File Transfer Protocol, **RFC 959**): protocolo para **transferir ficheros** entre ordenadores. Está **diseñado para ser usado por programas**.

- Permite **compartir ficheros** (binarios o de datos).
- Facilita el **uso de ordenadores remotos**.
- **Protege al usuario** de las variaciones en el sistema de ficheros de los distintos hosts.
- **Transfiere ficheros** real y eficientemente.

FTP: arquitectura con dos conexiones



La conexión de datos se abre solo para transferir (PORT o PASV) y se cierra después.

Dos conexiones: **control** (puerto 21) y **datos** (puerto 20).

Pieza	Qué hace
User-PI / Server-PI (Protocol Interpreter)	Intercambian los **comandos FTP** que fijan los parámetros de la comunicación (puerto, modo de transferencia, tipo de representación, estructura…). El Server-PI **escucha en el puerto 21** (*well-known port*). Es la **conexión de control**.
User-DTP / Server-DTP (Data Transfer Process)	Controlan el **intercambio de ficheros** (binarios o no). Usan el **puerto 20**. Es la **conexión de datos**.

Variante «esquema II»: un cliente controla **dos servidores** (A y B) y hace que se manden el fichero directamente entre ellos:

```
C->A : Connect                                  C->B : Connect
C->A : PASV
A->C : 227 Entering Passive Mode. A1,A2,A3,A4,a1,a2
                                                C->B : PORT A1,A2,A3,A4,a1,a2
                                                B->C : 200 Okay
C->A : STOR                                     C->B : RETR
         B->A : Connect to HOST-A, PORT-a       (B conecta directamente con A para los datos)
```

FTP: representación, estructura y modos

Aspecto	Opciones	Por defecto
Representación (`TYPE`)	**ASCII** (A, 8 bits; ficheros de texto) · EBCDIC (E) · **IMAGE** (I; binarios; grupos de 8 bits que se almacenan como bits contiguos) · LOCAL (el emisor define el tamaño del byte lógico)	ASCII
Control de formato (para ASCII/EBCDIC)	NON PRINT (sin control vertical) · TELNET FORMAT CONTROLS (CR, LF, NL…) · CARRIAGE CONTROL (ASA)	NON PRINT
Estructura del fichero (`STRU`)	**File** (secuencia de bytes sin estructura) · Record (secuencia de registros) · Page (ficheros discontinuos, con páginas con cabecera)	File
Modo de transmisión (`MODE`)	**Stream** (flujo de bytes) · Block (bloques con cabecera) · Compressed (datos normales, comprimidos y de control)	Stream

> Diferentes sistemas representan los datos de forma distinta: se usa una **representación estándar** en la comunicación y se reconvierte en los extremos.

FTP: los comandos

Son **cadenas de texto acabadas en CRLF**. **Todo comando tiene al menos una respuesta** asociada y algunos deben ir en grupo (p. ej. `RNFR` seguido de `RNTO`).

Grupo	Comando	Sintaxis y función
Control de acceso	USER	`USER <nombre>` especifica el usuario
	PASS	`PASS <contraseña>` especifica la contraseña
	ACCT	`ACCT <info>` identifica la cuenta
	CWD	`CWD <ruta>` cambia el directorio de trabajo
	CDUP	sube un directorio
	SMNT	cambia de sistema de ficheros sin cambiar login/password
	REIN	**cierra la sesión sin cerrar la conexión de control**
	QUIT	**cierra la sesión cerrando la conexión de control**
Parámetros de transferencia	PORT	`PORT h1,h2,h3,h4,p1,p2` indica host y puerto de datos
	PASV	pone el Server-DTP **en escucha** (modo pasivo)
	TYPE / STRU / MODE	fijan representación (A, E, I…), estructura y modo

Grupo	Comando	Sintaxis y función
Servicio	RETR	`RETR <ruta>` el servidor **envía** una copia del fichero (**descargar**)
	STOR	`STOR <ruta>` el servidor **recibe y almacena** un fichero (**subir**)
	STOU / APPE	guarda con nombre único generado por el servidor / **añade** al final del fichero
	REST	indica desde dónde reanudar la descarga (seguido de RETR)
	RNFR + RNTO	renombrar: primero el nombre antiguo, luego el nuevo
	ABOR	aborta la operación actual
	DELE / RMD / MKD	borrar fichero / borrar directorio / crear directorio
	PWD	muestra el directorio de trabajo actual
	LIST / NLST	lista ficheros (con información / solo nombres)
	SITE / SYST / STAT / HELP / NOOP	parámetros del host / sistema operativo / estado / ayuda / no hacer nada

FTP: el comando PORT y cómo se calcula

`PORT h1,h2,h3,h4,p1,p2`: la **IP** es `h1.h2.h3.h4`. El **puerto** es un número de **16 bits** que se reparte en dos números de 8 bits: **p1 = los 8 bits de más peso** y **p2 = los 8 de menos peso**.

> **Puerto = p1 × 256 + p2** · y al revés: **p1 = puerto ÷ 256 (entero)**, **p2 = resto**.

Puerto	Cálculo	PORT
4096	4096 = 2^{12} → p1 = 16, p2 = 0 (16 × 256 + 0)	`PORT 127,0,0,1,16,0`
5000	5000 ÷ 256 = 19 resto 136 → p1 = 19, p2 = 136	`PORT 192,168,1,10,19,136`
21	21 ÷ 256 = 0 resto 21 → p1 = 0, p2 = 21	`PORT 127,0,0,1,0,21`

> **Extra · activo vs pasivo**: en modo **activo** el cliente manda `PORT` y es el **servidor** quien abre la conexión de datos hacia el cliente. En modo **pasivo** el cliente manda `PASV` y el **servidor** queda «en escucha» y responde con `227 Entering Passive Mode …` (IP y puerto con el mismo formato) para que el **cliente** abra la conexión de datos.

FTP: respuestas y códigos

Cada respuesta tiene **dos partes**: un **código numérico de 3 dígitos** y un **texto descriptivo** para el humano (el texto **no es estándar**; lo que vale es el código). Una respuesta puede ser **multilínea**: `123-primera línea` … y la última `123 última línea` (con espacio en vez de guion).

Primer dígito	Significado	Ejemplo
1yz	Respuesta preliminar positiva (empezó, espera más)	150 Opening ASCII mode data connection for /bin/ls.
2yz	Comando ejecutado correctamente	200 Command okay
3yz	Respuesta intermedia positiva (falta información)	331 User name okay, need password
4yz	Respuesta «temporalmente» negativa (puede reintentarse)	421 Service not available, closing control connection
5yz	Respuesta definitivamente negativa	501 Syntax error in parameters or arguments

Segundo dígito	Tema
x0z	Errores de sintaxis
x1z	Respuestas a peticiones de información

Segundo dígito	Tema
x2z	Respuestas a peticiones de conexión
x3z	Proceso de autenticación
x4z	Sin especificar
x5z	Relacionadas con el sistema de ficheros

Implementación mínima: representación ASCII Non-print, modo stream, estructura File y Record y los comandos `user, quit, port, type, mode, stru, retr, stor, noop`.

FTP: una sesión real explicada línea a línea

```
220-SIA FTP service                     <- 220: servicio listo (respuesta multilínea con guion)
220 :-)Please enter your user name.
FTP Command>USER sia01                  -> 331 User name okay, Need password      (3yz: falta la contraseña)
FTP Command>PASS knock                  -> 230 User logged in.                    (2yz: correcto)
FTP Command>pwd                         -> 257 "/" is current directory.          (el servidor entiende pwd)
FTP Command>cd pub                      -> 500 'cd pub': command not understood.  (5yz: «cd» NO es un comando FTP)
FTP Command>CWD pub                     -> 250 "/pub" is current directory        (el comando FTP correcto es CWD)
FTP Command>RETR prueba.txt             -> 550 Access denied                      (5yz, x5z: sistema de ficheros/permisos)
FTP Command>QUIT                        -> 221 Goodbye. Control connection closed.
```

- Mira el error de `cd pub`: **«cd» no es un comando del protocolo FTP**; el comando correcto para cambiar de directorio es `CWD`. El servidor responde con un 5yz («command not understood») y se sigue conectado.
- **Descargar = RETR; subir = STOR**.

7 El servidor multihilo del examen: receta, plantillas y eje

Qué te piden siempre

El ejercicio de **desarrollo (5 puntos)** es siempre de la misma familia: *«el equipo está desarrollando un sistema al que se conectan clientes mediante **Sockets**; implementa **el hilo de ejecución del servidor que gestiona los comandos recibidos**»*. Te dan una tabla de **comandos de texto** (`ALTAEMPLEADO id nombre`, `ENTRADA id`…) con las respuestas exactas que debe devolver, y reglas de negocio («no se puede fichar entrada si ya hay una abierta»). Tú escribes **el hilo** (y quizá una clase de datos).

> Idea central: **un cliente = un hilo = un socket**. El hilo lee una línea, la interpreta (`split` + `switch`), ejecuta la regla de negocio y escribe una línea de respuesta. Se repite hasta que el cliente se va.

Receta de 7 pasos

1. **Atributos y constructor**: `private Socket socket;` (y `private Servidor servidor;` si hay estado compartido) + constructor que los recibe. La clase `extends Thread`.
2. **run()** con `try { … } catch (IOException e)`: aquí van los **buffers**: `BufferedReader br` sobre `socket.getInputStream()` y `PrintWriter pw` sobre `socket.getOutputStream()`.
3. **Bucle de lectura**: `while ((linea = br.readLine()) != null)` (o `while(!salir)` si hay comando `EXIT`).
4. **Trocear** la línea: `String[] partes = linea.split(" ");` → `partes[0]` es el comando. Si algún campo lleva **espacios** (nombre completo, descripción), usa `split(" ", 3)` o `substring`+`indexOf`.
5. `switch (partes[0])` con un `case` por comando y un `default` («Comando no reconocido»).
6. En cada caso: **comprobar** `partes.length`, que el objeto **exista** (empleado, matrícula…) y las **reglas** del enunciado; construir la **respuesta con el texto exacto** que pide el enunciado.
7. **Responder**: `pw.println(respuesta); pw.flush();`. Al terminar (EXIT o `null`), **cerrar el socket**.

Plantilla 1 · Servidor (casi siempre igual)

Solo hace `accept()` en bucle y lanza un hilo por cliente. **Puerto: el que diga el enunciado**; si no dice nada, pon uno (8000, 5050…).

```
import java.io.IOException;
import java.net.ServerSocket;
import java.net.Socket;

public class Servidor {
    public static final int PUERTO = 5050;

    public void ejecutar() {
        try {
            ServerSocket serverSocket = new ServerSocket(Servidor.PUERTO);
            System.out.println("Servidor lanzado esperando clientes...");
            while (true) {
                Socket socket = serverSocket.accept();     // se bloquea hasta que llega un cliente
                HiloServidor hilo = new HiloServidor(socket);
                hilo.start();                              // el main vuelve al accept() enseguida
            }
        } catch (IOException e) {
            e.printStackTrace();
        }
    }

    public static void main(String[] args) {
        (new Servidor()).ejecutar();
    }
}
```

Plantilla 2 · Hilo con switch (aquí está todo lo que cambia)

```
import java.io.BufferedReader;
import java.io.IOException;
import java.io.InputStreamReader;
import java.io.OutputStreamWriter;
import java.io.PrintWriter;
import java.net.Socket;

public class HiloServidor extends Thread {
    private Socket socket;

    public HiloServidor(Socket socket) {
        this.socket = socket;
    }
    @Override
    public void run() {
        try {
            BufferedReader br = new BufferedReader(new InputStreamReader(socket.getInputStream()));
            PrintWriter pw = new PrintWriter(new OutputStreamWriter(socket.getOutputStream()));
            boolean salir = false;
            while (!salir) {
                String linea = br.readLine();
                if (linea == null) break;                  // el cliente cerro la conexion
                String[] partes = linea.split(" ");
                String respuesta = "";
                switch (partes[0].toUpperCase()) {
                case "SUMA":
                    respuesta = "" + (Integer.parseInt(partes[1]) + Integer.parseInt(partes[2]));
                    break;
                case "RESTA":
                    respuesta = "" + (Integer.parseInt(partes[1]) - Integer.parseInt(partes[2]));
                    break;
                case "MULTIPLICA":
                    respuesta = "" + (Integer.parseInt(partes[1]) * Integer.parseInt(partes[2]));
                    break;
                case "EXIT":
                    respuesta = "Adios";
                    salir = true;
                    break;
                default:
                    respuesta = "Comando no reconocido";
                }
                pw.println(respuesta);
                pw.flush();
            }
            socket.close();
        } catch (IOException | NumberFormatException | ArrayIndexOutOfBoundsException e) {
            System.out.println("Conexion finalizada: " + e);
        }
    }
}
```

Qué hace cada trozo:

- `public class HiloServidor extends Thread` + atributo `socket` + constructor: guarda el socket de **su** cliente.
- `BufferedReader br …; PrintWriter pw …`: los canales de lectura y escritura **con ese cliente**.
- `boolean salir = false;` y `while (!salir)`: el bucle sigue **mientras no se escriba EXIT**.
- `String linea = br.readLine(); if (linea == null) break;`: lee una línea; si es `null` el cliente se desconectó y salimos para no quedarnos en bucle infinito.
- `String[] partes = linea.split(" ")` y `switch (partes[0].toUpperCase())`: elige el comando **sin distinguir mayúsculas**.
- `Integer.parseInt(partes[1])`: convierte el texto «2» en el número 2. Si no es un número lanza `NumberFormatException` (por eso va dentro del `try/catch`).
- `respuesta` se rellena en cada `case`; al final una sola vez `pw.println(respuesta); pw.flush();`.
- `socket.close()` al salir del bucle libera la conexión.

Plantilla 3 · Cliente (con teclado)

Se hace rápido y **da puntos**. El **host/puerto deben coincidir** con los del servidor (`127.0.0.1` = esta máquina); termina cuando el usuario escribe `EXIT`. Lo único que cambia entre ejercicios es el **texto del menú**.

```
import java.io.BufferedReader;
import java.io.IOException;
import java.io.InputStreamReader;
import java.io.OutputStreamWriter;
import java.io.PrintWriter;
import java.net.Socket;
import java.util.Scanner;

public class Cliente {
    public void ejecutar() {
        try (Socket socket = new Socket("127.0.0.1", Servidor.PUERTO)) {
            PrintWriter pw = new PrintWriter(new OutputStreamWriter(socket.getOutputStream()));
            BufferedReader br = new BufferedReader(new InputStreamReader(socket.getInputStream()));
            Scanner teclado = new Scanner(System.in);
            String linea = "";
            System.out.println("Comandos: SUMA a b | RESTA a b | MULTIPLICA a b | EXIT");
            while (!linea.equalsIgnoreCase("EXIT")) {
                linea = teclado.nextLine();
                pw.println(linea);
                pw.flush();
                System.out.println(br.readLine());
            }
        } catch (IOException e) {
            e.printStackTrace();
        }
    }

    public static void main(String[] args) {
        (new Cliente()).ejecutar();
    }
}
```

Ejemplo resuelto 1 · Calculadora (ejemplo del profesor)

Comandos: `SUMA a b`, `RESTA a b`, `MULTIPLICA a b` y `EXIT`. Es la plantilla anterior tal cual. Traza de una sesión:

```
Cliente escribe:  SUMA 2 3        -> partes = ["SUMA","2","3"]  -> "5"
Cliente escribe:  MULTIPLICA 4 5  -> partes = ["MULTIPLICA","4","5"] -> "20"
Cliente escribe:  FOO             -> default  -> "Comando no reconocido"
Cliente escribe:  EXIT            -> respuesta "Adios", salir = true, se cierra el socket
```

> Cambios respecto a la plantilla básica que el profesor hizo: (1) en el cliente, el texto del menú; (2) en el hilo, `String partes[]`, `partes = linea.split(" ")`, un **switch** sobre `partes[0]` y `Integer.parseInt` para convertir String a entero.

Ejemplo resuelto 2 · Multas con Hashtable (examen de otros años)

Gestiona **multas por matrícula**. Aquí aparece lo nuevo: **los hilos deben compartir los datos** de todos los clientes. Se hace guardando una `Hashtable<String, Matricula>` en el **Servidor** y **pasando el servidor a cada hilo**.

1. **Matricula** (clase de datos): `ArrayList<Integer>` con las multas; `getCuantiaMultas()` suma todas (como sumar la cesta en POO); `addMulta(Integer)`; `addPago(int)`; `getPendiente()` = multas − pagado.
2. **Servidor**: atributo `Hashtable` y métodos `getMatricula`, `addMatricula`, `removeMatricula` para que los hilos accedan. En `accept()` se hace `new HiloServidor(socket, this)` (**this** = el servidor).
3. **HiloServidor**: guarda `servidor` y `socket`; `boolean salir`; actualiza `matricula = servidor.getMatricula(partes[1])` en cada comando; el `switch` aplica ALTA / MULTA / PAGO / PENDIENTE / BORRAR; **«if (partes.length == 3 && matricula != null)»** para exigir 3 datos y que exista.

```
import java.util.ArrayList;

public class Matricula {
    private String numero;
    private ArrayList<Integer> multas = new ArrayList<Integer>();
    private int pagado = 0;

    public Matricula(String numero) {
        this.numero = numero;
    }
    public String getNumero() { return numero; }

    public void addMulta(Integer cuantia) { multas.add(cuantia); }
    public int getCuantiaMultas() {            // suma de todas las multas
        int total = 0;
        for (Integer m : multas) total += m;
        return total;
    }
    public void addPago(int cantidad) { pagado += cantidad; }

    public int getPendiente() { return getCuantiaMultas() - pagado; }
}
```

```
import java.io.IOException;
import java.net.ServerSocket;
import java.net.Socket;
import java.util.Hashtable;

public class Servidor {
    public static final int PUERTO = 5050;
    // Hashtable es thread-safe: todos los hilos comparten esta unica tabla
    private Hashtable<String, Matricula> matriculas = new Hashtable<String, Matricula>();

    public Matricula getMatricula(String numero) { return matriculas.get(numero); }

    public String addMatricula(String numero) {
        if (matriculas.containsKey(numero)) return "Error. La matricula ya existe";
        matriculas.put(numero, new Matricula(numero));
        return "Ok. Matricula insertada";
    }

    public String removeMatricula(Matricula matricula) {
        if (matricula == null) return "Error. La matricula no existe";
        matriculas.remove(matricula.getNumero());
        return "Ok. Matricula borrada";
    }

    public void ejecutar() {
        try (ServerSocket serverSocket = new ServerSocket(PUERTO)) {
            while (true) {
                Socket socket = serverSocket.accept();
                new HiloServidor(socket, this).start();    // se pasa el servidor al hilo
            }
        } catch (IOException e) {
            e.printStackTrace();
        }
    }

    public static void main(String[] args) {
        (new Servidor()).ejecutar();
    }
}
```

```
import java.io.BufferedReader;
import java.io.IOException;
import java.io.InputStreamReader;
import java.io.OutputStreamWriter;
import java.io.PrintWriter;
import java.net.Socket;

public class HiloServidor extends Thread {
    private Socket socket;
    private Servidor servidor;
    private boolean salir = false;

    public HiloServidor(Socket socket, Servidor servidor) {
        this.socket = socket;
```

```
        this.servidor = servidor;
    }

    @Override
    public void run() {
        try (BufferedReader br = new BufferedReader(new InputStreamReader(socket.getInputStream()));
                PrintWriter pw = new PrintWriter(new OutputStreamWriter(socket.getOutputStream()))) {
            while (!salir) {
                String lineaLeida = br.readLine();
                if (lineaLeida == null) break;
                String[] partes = lineaLeida.split(" ");
                String respuesta = "";
                Matricula matricula = null;
                if (partes.length > 1) matricula = servidor.getMatricula(partes[1]);
                switch (partes[0].toUpperCase()) {
                case "ALTA":
                    respuesta = partes.length == 2 ? servidor.addMatricula(partes[1]) : "Error. Uso: ALTA matricula";
                    break;
                case "MULTA":
                    if (partes.length == 3 && matricula != null) {
                        matricula.addMulta(Integer.parseInt(partes[2]));
                        respuesta = "Ok. Multa anadida";
                    } else respuesta = "Error. Uso: MULTA matricula cuantia (la matricula debe existir)";
                    break;
                case "PAGO":
                    if (partes.length == 3 && matricula != null) {
                        matricula.addPago(Integer.parseInt(partes[2]));
                        respuesta = "Ok. Pago registrado";
                    } else respuesta = "Error. Uso: PAGO matricula cantidad (la matricula debe existir)";
                    break;
                case "PENDIENTE":
                    respuesta = matricula != null ? "Pendiente: " + matricula.getPendiente() + " euros"
                            : "Error. La matricula no existe";
                    break;
                case "BORRAR":
                    respuesta = servidor.removeMatricula(matricula);
                    break;
                case "EXIT":
                    respuesta = "Adios";
                    salir = true;
                    break;
                default:
                    respuesta = "Comando no reconocido";
                }
                pw.println(respuesta);
                pw.flush();
            }
        } catch (IOException | NumberFormatException e) {
            System.out.println("Conexion finalizada: " + e);
        }
    }
}
```

```
ALTA 1234ABC          -> Ok. Matricula insertada
ALTA 1234ABC          -> Error. La matricula ya existe
MULTA 1234ABC 200     -> Ok. Multa anadida
MULTA 1234ABC 100     -> Ok. Multa anadida
PAGO 1234ABC 120      -> Ok. Pago registrado
PENDIENTE 1234ABC     -> Pendiente: 180 euros      (200 + 100 - 120)
BORRAR 1234ABC        -> Ok. Matricula borrada
```

Ejemplo resuelto 3 · Materiales (y matar hilos con EXIT)

Igual que multas, pero además el **Servidor guarda los hilos** en un `ArrayList<HiloServidor>` con un **id** numérico para poder cerrarlos. Con `EXIT`, el hilo llama a `servidor.matarHilo(id)`, que busca el hilo por id, llama a su `cerrarSocket()` y lo quita de la lista. La **descripción puede tener espacios**, por eso se usa `substring` + `indexOf`: se queda con *todo* lo que va tras la cantidad.

```
public class Material {
    private String id;
    private String descripcion;
    private int cantidad;

    public Material(String id, String descripcion, int cantidad) {
        this.id = id;
        this.descripcion = descripcion;
        this.cantidad = cantidad;
    }
    public String getId() { return id; }
    public int getCantidad() { return cantidad; }

    public boolean addReserva(int unidades) {      // resta unidades si hay stock
        if (unidades <= 0 || unidades > cantidad) return false;
        cantidad -= unidades;
        return true;
    }

    @Override
    public String toString() {
        return id + " - " + descripcion + " (" + cantidad + " uds)";
    }
}
```

```
import java.io.IOException;
import java.net.ServerSocket;
import java.net.Socket;
import java.util.ArrayList;
import java.util.Hashtable;

public class Servidor {
    public static final int PUERTO = 5050;
    private Hashtable<String, Material> materiales = new Hashtable<String, Material>();
    private ArrayList<HiloServidor> hilos = new ArrayList<HiloServidor>();   // para poder "matar" hilos
    private int contadorHilos = 0;

    public synchronized String addMaterial(String id, String descripcion, int cantidad) {
        if (materiales.containsKey(id)) return "Error. El material ya existe";
        materiales.put(id, new Material(id, descripcion, cantidad));
        return "Ok. Material insertado";
    }
    public Material getMaterial(String id) { return materiales.get(id); }

    public synchronized String listar() {
        StringBuilder sb = new StringBuilder();
        for (Material m : materiales.values()) sb.append(m).append(" | ");
        return sb.length() == 0 ? "No hay materiales" : sb.toString();
    }

    public synchronized void matarHilo(int idHilo) {
        for (HiloServidor h : hilos) {
            if (h.getIdHilo() == idHilo) {
                h.cerrarSocket();
                hilos.remove(h);
                break;
            }
        }
    }

    public void ejecutar() {
        try (ServerSocket serverSocket = new ServerSocket(PUERTO)) {
            while (true) {
                Socket socket = serverSocket.accept();
                HiloServidor hilo = new HiloServidor(socket, this, contadorHilos++);
                synchronized (this) { hilos.add(hilo); }
                hilo.start();
            }
        } catch (IOException e) {
            e.printStackTrace();
        }
    }

    public static void main(String[] args) {
        (new Servidor()).ejecutar();
    }
}
```

```
import java.io.BufferedReader;
import java.io.IOException;
import java.io.InputStreamReader;
import java.io.OutputStreamWriter;
import java.io.PrintWriter;
import java.net.Socket;

public class HiloServidor extends Thread {
    private Socket socket;
    private Servidor servidor;
    private int idHilo;
    private boolean salir = false;

    public HiloServidor(Socket socket, Servidor servidor, int idHilo) {
        this.socket = socket;
        this.servidor = servidor;
        this.idHilo = idHilo;
    }
    public int getIdHilo() { return idHilo; }

    public void cerrarSocket() {
        try { socket.close(); } catch (IOException e) { }
    }

    @Override
    public void run() {
        try {
            BufferedReader br = new BufferedReader(new InputStreamReader(socket.getInputStream()));
            PrintWriter pw = new PrintWriter(new OutputStreamWriter(socket.getOutputStream()));
            while (!salir) {
                String lineaLeida = br.readLine();
                if (lineaLeida == null) break;
                String[] partes = lineaLeida.split(" ");
                String respuesta = "";
                switch (partes[0].toUpperCase()) {
                case "ALTA": {   // ALTA id cantidad descripcion con espacios...
                    if (partes.length < 4) { respuesta = "Error. Uso: ALTA id cantidad descripcion"; break; }
                    // substring + indexOf: coge todo lo que hay tras el 3er espacio
                    int inicio = lineaLeida.indexOf(partes[2]) + partes[2].length() + 1;
                    String descripcion = lineaLeida.substring(inicio);
                    respuesta = servidor.addMaterial(partes[1], descripcion, Integer.parseInt(partes[2]));
                    break;
                }
                case "RESERVA": {
                    Material m = partes.length == 3 ? servidor.getMaterial(partes[1]) : null;
                    if (m == null) respuesta = "Error. Uso: RESERVA id unidades (el material debe existir)";
                    else respuesta = m.addReserva(Integer.parseInt(partes[2])) ? "Ok. Reserva realizada" : "Error. Stock insuficiente";
                    break;
                }
                case "LISTAR":
                    respuesta = servidor.listar();
                    break;
                case "EXIT":
                    respuesta = "Adios";
                    salir = true;
                    break;
                default:
                    respuesta = "Comando no reconocido";
                }
                pw.println(respuesta);
                pw.flush();
            }
            servidor.matarHilo(this.idHilo);    // EXIT: el servidor cierra el socket y borra el hilo
        } catch (IOException | NumberFormatException e) {
            System.out.println("Conexion finalizada: " + e);
        }
    }
}
```

Congresos (tercer ejercicio de otros años) es lo mismo con una clase `Congreso` y un comando **LISTAR**: el servidor devuelve (una copia de) la `Hashtable` y el hilo recorre los valores concatenando sus `toString()`; **BORRAR id** llama a `servidor.removeCongreso(id)`.

Ejemplo resuelto 4 · El parcial real: fichaje de empleados

Comandos: `ALTAEMPLEADO id nombreCompleto`, `ENTRADA id`, `SALIDA id`, `HORAS id`, `ESTADO id`. **Reglas**: no se puede *ENTRADA* si ya hay entrada abierta; no se puede *SALIDA* si no la hay. Existe una clase `GestionFechas` con `getHoraActual()` (Timestamp), `getHorasEntre(inicio, fin)` (float) y `getTexto(Timestamp)` (String).

Cómo pensarlo: cada empleado tiene un *estado* (¿entrada abierta? ¿desde cuándo?) y unas *horas acumuladas*. ENTRADA guarda la hora actual; SALIDA suma `getHorasEntre(entrada, ahora)` y borra la entrada abierta; ESTADO consulta; HORAS devuelve el acumulado.

Clase de datos Empleado:

```
import java.sql.Timestamp;

public class Empleado {
    private String id;
    private String nombre;
    private Timestamp entrada;      // null si no hay entrada abierta
    private float horasAcumuladas = 0;

    public Empleado(String id, String nombre) {
        this.id = id;
        this.nombre = nombre;
    }
    public String getId() { return id; }
    public String getNombre() { return nombre; }
    public Timestamp getEntrada() { return entrada; }
    public void setEntrada(Timestamp entrada) { this.entrada = entrada; }
    public boolean tieneEntradaAbierta() { return entrada != null; }
    public float getHorasAcumuladas() { return horasAcumuladas; }
    public void sumaHoras(float horas) { this.horasAcumuladas += horas; }
}
```

Clase pedida: HiloGestionFichaje (una excepción por comando y mensajes **idénticos al enunciado**):

```
import java.io.BufferedReader;
import java.io.IOException;
import java.io.InputStreamReader;
import java.io.OutputStreamWriter;
import java.io.PrintWriter;
import java.net.Socket;
import java.sql.Timestamp;
import java.util.Hashtable;

public class HiloGestionFichaje extends Thread {
    // Listado de empleados (compartido por todos los hilos -> accesos synchronized)
    private static Hashtable<String, Empleado> empleados = new Hashtable<String, Empleado>();

    private Socket socket;
    private BufferedReader br;
    private PrintWriter pw;

    public HiloGestionFichaje(Socket socket) {
        this.socket = socket;
    }

    @Override
    public void run() {
        try {
            // Buffers de comunicacion con el cliente
            this.br = new BufferedReader(new InputStreamReader(this.socket.getInputStream()));
            this.pw = new PrintWriter(new OutputStreamWriter(this.socket.getOutputStream()));
            String linea;
            while ((linea = this.br.readLine()) != null) {
                String respuesta = this.procesar(linea);
                this.pw.println(respuesta);
                this.pw.flush();
            }
        } catch (IOException e) {
            System.out.println("Conexion finalizada");
        } finally {
            try { this.socket.close(); } catch (IOException e) { }
        }
    }

    private String procesar(String linea) {
```

```
        String[] partes = linea.trim().split(" ", 3);   // 3: el nombre completo puede llevar espacios
        String comando = partes[0].toUpperCase();

        switch (comando) {
        case "ALTAEMPLEADO":
            if (partes.length < 3) return "Error. Uso: ALTAEMPLEADO idEmpleado nombreCompleto";
            return this.altaEmpleado(partes[1], partes[2]);
        case "ENTRADA":
            if (partes.length < 2) return "Error. Uso: ENTRADA idEmpleado";
            return this.entrada(partes[1]);
        case "SALIDA":
            if (partes.length < 2) return "Error. Uso: SALIDA idEmpleado";
            return this.salida(partes[1]);
        case "HORAS":
            if (partes.length < 2) return "Error. Uso: HORAS idEmpleado";
            return this.horas(partes[1]);
        case "ESTADO":
            if (partes.length < 2) return "Error. Uso: ESTADO idEmpleado";
            return this.estado(partes[1]);
        default:
            return "Error. Comando no reconocido";
        }
    }

    private synchronized String altaEmpleado(String id, String nombre) {
        if (empleados.containsKey(id)) {
            return "Error. El empleado ya existe";
        }
        empleados.put(id, new Empleado(id, nombre));
        return "Ok. Empleado insertado";
    }

    private synchronized String entrada(String id) {
        Empleado e = empleados.get(id);
        if (e == null) return "Error. El empleado no existe";
        if (e.tieneEntradaAbierta()) {
            return "Error. El empleado ya ha registrado la entrada";
        }
        Timestamp ahora = GestionFechas.getHoraActual();
        e.setEntrada(ahora);
        return "Ok. Entrada registrada. " + GestionFechas.getTexto(ahora);
    }
    private synchronized String salida(String id) {
        Empleado e = empleados.get(id);
        if (e == null) return "Error. El empleado no existe";
        if (!e.tieneEntradaAbierta()) {
            return "Error. El empleado no tiene registrada entrada";
        }
        Timestamp ahora = GestionFechas.getHoraActual();
        e.sumaHoras(GestionFechas.getHorasEntre(e.getEntrada(), ahora));
        e.setEntrada(null);                              // cierra la entrada abierta
        return "Ok. Salida registrada. " + GestionFechas.getTexto(ahora);
    }

    private synchronized String horas(String id) {
        Empleado e = empleados.get(id);
        if (e == null) return "Error. El empleado no existe";
        return "Ok. " + e.getNombre() + " tiene trabajadas " + e.getHorasAcumuladas() + " horas.";
    }

    private synchronized String estado(String id) {
        Empleado e = empleados.get(id);
        if (e == null) return "Error. El empleado no existe";
        if (e.tieneEntradaAbierta()) {
            return "El empleado ha registrado entrada: " + GestionFechas.getTexto(e.getEntrada());
        }
        return "El empleado no ha registrado entrada";
    }
}
```

Qué mira el corrector: buffers bien creados + `flush` · bucle de lectura · `split` y `switch` · alta rechaza duplicados · ENTRADA y SALIDA respetan sus reglas · uso correcto de `GestionFechas` · mensajes exactos · comprobar si el empleado existe.

> El listado de empleados es `static` y los métodos son `synchronized` porque lo comparten **todos los hilos** (todos los clientes). Si el enunciado dice «el listado se podrá almacenar como atributo en la clase implementada», `static` es la forma de que lo vean todos.

Reglas de oro para los comandos de texto

- **Siempre** comprueba `partes.length` antes de usar `partes[1]`/`partes[2]` (si no, `ArrayIndexOutOfBoundsException`).
- **Siempre** comprueba que el objeto existe (empleado, matrícula, libro…) antes de operar con él.
- **Siempre** un `default` en el `switch`.
- Los mensajes de respuesta deben ser **exactamente** los del enunciado (mayúsculas, puntos, tildes).
- Todo lo compartido entre clientes (la tabla de datos) va en el **Servidor** o en un atributo `static`; las operaciones «comprobar y luego insertar» en métodos `synchronized`.
- `pw.flush()` después de **cada** `println`.
- Cierra el socket al terminar (EXIT, `null` o excepción).

Distribución del tiempo en el examen

Parte	Tiempo orientativo	Consejo
Preguntas cortas	≈ 20 min	Respuestas con las **palabras clave** de los apuntes. Escribe listas, no párrafos.
Programación (4 preguntas)	≈ 20 min	Identifica el **patrón** (start/run, Runnable sin Thread, carrera, BindException) y da causa + arreglo.
Desarrollo (5 puntos)	≈ 50 min	Esqueleto del hilo primero (pasos 1–3), luego un comando cada vez. Reserva 5 min para repasar mensajes y `flush`.

8 Errores típicos que caen en el examen

Preguntas de programación del parcial y cómo contestarlas

Si ves…	Causa	Cómo se arregla
`BindException: Address already in use` al ejecutar dos veces seguidas	La primera ejecución **sigue viva** (servidor en `while(true)` que nunca termina ni cierra el `ServerSocket`) y mantiene el puerto ocupado.	Parar la anterior, cambiar de puerto o cerrar el `ServerSocket`. También salir del bucle cuando `readLine()` devuelve `null`.
Se lanzan 10 hilos con `(new Thread()).start()` y **no sale «Hola Mundo»**	El `Thread` se crea **sin pasarle el Runnable**: su `run()` está vacío.	`(new Thread(miHilo)).start()`.
5 hilos × 1000 incrementos y el contador **no llega a 5000**	**Condición de carrera**: comparten el mismo objeto y `contador++` no es atómico.	`synchronized` o `AtomicInteger`.
`IllegalThreadStateException` en `Thread.start()`	`start()` llamado **varias veces sobre el mismo objeto**.	Crear un objeto nuevo en cada vuelta del bucle.
Un segundo cliente conecta pero **no recibe respuesta**	Servidor de **un solo hilo**, atrapado en el bucle del primer cliente.	Un hilo por cliente; el main solo hace `accept()`.
`ConnectException: Connection refused`	El servidor no está arrancado o el puerto/IP no coinciden.	Arrancar antes el servidor; mismo puerto.
`t.run()` imprime «main»	`run()` es un método normal.	Usar `start()`.
`readLine()` devuelve `null` y el bucle imprime «null» sin parar	El cliente cerró la conexión.	`while ((linea = br.readLine()) != null)`.

Orden de salida con join(): hilo que duerme 500 ms e imprime «A»; el main imprime «B», hace `h.join()` y luego «C» → **B, A, C**. Sin `join()` → **B, C, A**.

9 Glosario rápido

De la A a la Z

Accept()
Método de `ServerSocket` que espera (bloqueante) la llegada de un cliente y devuelve el `Socket` de esa conexión.

Asíncrono / no bloqueante
Operación que no espera a que termine; el programa sigue y se le avisa después.

Back-end / servidor
Programa que ofrece un servicio y espera peticiones.

Bind
Asociar un socket a un puerto del sistema operativo.

Bloqueo indefinido
Una operación bloqueante que nunca se desbloquea. Se evita con temporizadores o hilos.

BufferedReader
Clase que permite leer **líneas** enteras (`readLine()`) de un flujo de caracteres.

Cookie
Identificador que el servidor da al navegador para recordar al usuario, ya que HTTP no tiene estado.

CRLF
Retorno de carro + salto de línea (`\r\n`); termina las líneas de HTTP y FTP.

Condición de carrera
Resultado incorrecto por modificar varios hilos la misma variable a la vez sin sincronizar.

DTP
Data Transfer Process: el proceso FTP que transfiere los ficheros (conexión de datos, puerto 20).

Estado
Valores de los atributos de un objeto en un momento dado; una comunicación «con estado» recuerda interacciones anteriores.

Flush
Forzar el envío del contenido del buffer de un `PrintWriter`.

Hilo (thread)
Secuencia única de control de flujo dentro de un programa; unidad mínima ejecutable.

HTTP
Protocolo de transferencia de hipertexto: petición-respuesta de texto, sin estado.

IPC
Inter-Process Communication: comunicación entre procesos.

join()
Esperar a que termine otro hilo (opcionalmente con tiempo máximo).

Multicast / Unicast
De un proceso a varios / de un proceso a otro.

Paquete (package)
Carpeta que agrupa clases en Java (`edu.ucam.sockets`).

PI
Protocol Interpreter: el intérprete de comandos FTP (conexión de control, puerto 21).

Protocolo
Reglas acordadas para que dos procesos se entiendan.

Puerto
Número que identifica qué programa de una máquina recibe los datos.

Runnable
Interfaz con el método `run()`; se pasa a un `Thread` para ejecutarlo.

Socket
Punto de conexión entre dos procesos identificado por IP y puerto.

start()
Crea el hilo y ejecuta su `run()` en paralelo; solo una vez por objeto.

Síncrono / bloqueante
Operación que espera a terminar antes de continuar.

synchronized
Palabra reservada que permite que solo un hilo a la vez ejecute un método/bloque.

Thread.sleep(ms)
Duerme el hilo actual los milisegundos indicados.

URL
Universal Resource Locator: identifica un recurso al que se puede pedir un método.

Workspace
Carpeta de Eclipse donde se guardan los proyectos.

10 Preguntas de teoría con respuesta modelo

Para repasar el día antes

Cubre la respuesta, contesta en voz alta y compara. Están escritas como se contestarían en el examen.

1. ¿Qué es un hilo y para qué se suele utilizar?

Un hilo de ejecución es la secuencia única de control de flujo dentro de un programa; es la unidad de código más pequeña que se puede ejecutar. Se utiliza para ejecutar tareas lentas, no paralizar el programa principal mientras se hacen otras tareas, evitar bloqueos inaceptables o indefinidos y manejar varias tareas a la vez (por ejemplo, un servidor que atiende a varios clientes).

2. Ventajas de la computación distribuida.

Es económica (los computadores y el acceso a red son baratos), permite compartir recursos, es escalable (se añaden máquinas según crece la carga) y tolera fallos (si un nodo cae, otros siguen). Como inconvenientes: múltiples puntos de fallo y aspectos de seguridad.

3. Características de HTTP y estructura de un paquete.

HTTP (RFC 1945) es un protocolo de texto de solicitud-respuesta basado en URL, usado por el 100 % de usuarios y servidores, genérico, que no es lo mismo que HTML y que no tiene estado (se resuelve con cookies). Un mensaje tiene primera línea (método + recurso + versión en la petición; versión + código + frase en la respuesta), cabeceras campo:valor, una línea en blanco CRLF y el cuerpo.

4. ¿Qué es un socket y cómo se activa?

Es el punto de conexión entre dos procesos, identificado por una IP y un puerto. En el servidor se activa con accept(), que espera hasta que un cliente se conecta.

5. ¿Cómo se crea y lanza un hilo en Java?

Heredando de Thread (class X extends Thread, implementar run() y llamar a start()) o implementando Runnable (implementar run() y lanzar con new Thread(obj).start()).

6. ¿Qué es un bloqueo indefinido y cómo se evita?

Ocurre cuando una petición espera una operación que nunca termina y no se desbloquea. Se evita con temporizadores (límite de espera) o con procesos hijo/hilos de ejecución que hacen la tarea lenta sin paralizar el programa.

7. ¿Por qué el contador no llega a 5000 con 5 hilos que suman 1000?

Porque los 5 hilos comparten el mismo objeto y contador++ no es atómico (leer, sumar, escribir): se pierden incrementos (condición de carrera). Se arregla con synchronized (o AtomicInteger).

8. ¿Por qué salta IllegalThreadStateException?

Porque se llama a start() más de una vez sobre el mismo objeto Thread; un hilo solo se puede arrancar una vez. Hay que crear un objeto nuevo por cada hilo.

9. ¿Por qué salta BindException: Address already in use?

Porque el puerto ya está ocupado, normalmente por una ejecución anterior del servidor que sigue viva (un while(true) que nunca termina y no cierra el ServerSocket).

10. ¿Qué es FTP y cuántas conexiones usa?

File Transfer Protocol (RFC 959) permite transferir ficheros entre ordenadores. Usa dos conexiones: control (comandos y respuestas, puerto 21) y datos (transferencia de ficheros, puerto 20).

11. Explica PORT 192,168,1,10,19,136.

Indica al servidor que abra la conexión de datos a la IP 192.168.1.10 y al puerto 19·256 + 136 = 5000.

12. ¿Qué significan los códigos FTP 2yz, 3yz y 5yz?

2yz: comando ejecutado correctamente (200 Command okay); 3yz: respuesta intermedia positiva que necesita más información (331 User name okay, need password); 5yz: respuesta negativa definitiva (550 Access denied).